Research: Segmentation Post-Processing & Morphological Operations Atoms

1. Introduction: The Criticality of Post-Processing in Segmentation Computational Directed Graphs

In the contemporary landscape of computer vision, the deployment of deep learning architectures for semantic and instance segmentation has yielded unprecedented analytical capabilities across diverse domains, spanning from high-resolution satellite imagery analysis to microscopic histological quantification. However, the raw output of a neural network—frequently taking the form of probabilistic maps or raw, uncalibrated logits—seldom represents the terminal, usable artifact required by downstream analytical engines. Neural networks, by their fundamental convolutional or attention-based nature, excel at capturing complex, high-level semantic features and global context but frequently struggle to resolve high-frequency spatial details. This limitation consistently manifests as outputs containing jagged, aliased boundaries, fragmented and disjointed instances, spurious artifacts, and severe topological inconsistencies. To bridge the definitive gap between raw, continuous tensor outputs and production-ready geometric or statistical representations, robust post-processing pipelines are strictly necessary.

These post-processing pipelines are optimally structured as Computational Directed Graphs (CDGs), where every discrete node represents an atomic, mathematically deterministic operation. This exhaustive research report investigates and codifies best-in-class, pure-function implementations for these critical post-processing atoms, targeting integration into the sciona-atoms-dl repository. The architectural constraints of the sciona-atoms-dl repository demand a strict functional programming paradigm: atoms must strictly consume defined inputs (e.g., multidimensional arrays, explicit scalar parameters) and return deterministic outputs without altering global state, managing opaque GPU memory states, or performing file input/output (I/O) operations. By defining these operations as mathematically pure functions, the CDG ensures perfect reproducibility across diverse compute environments, guaranteed thread safety during massive parallel map-reduce operations, and highly efficient CPU cache utilization during execution.

The research encapsulated herein spans multiple critical domains of image post-processing. It comprehensively details morphological signal filtering techniques designed to mathematically add or remove structural artifacts based on lattice theory. It further explores topological corrections that ensure manifold integrity, such as mathematical hole filling and connected component filtering based on geometric thresholds. For sub-pixel boundary alignment, the report dissects the mathematical formulation and integration of Dense Conditional Random Fields (CRFs). To resolve the overlapping instance problem inherent in semantic maps,

topographic watershed algorithms are evaluated. Furthermore, the report details Ramer-Douglas-Peucker (RDP) algorithms for discrete vector smoothing and high-performance, strictly vectorized Run-Length Encoding (RLE) implementations necessary for Kaggle-standard data serialization. Finally, domain-specific continuous operations, notably the Kulik et al. multi-spectral band arithmetic for satellite imagery false-color generation, are analyzed and codified into standard functional contracts.

Through an exhaustive algorithmic and performance analysis of standard scientific computing libraries, this report establishes the mathematical foundation, computational performance characteristics, and optimal pure-function contracts for thirteen distinct post-processing atoms.

2. Morphological Signal Filtering: Mathematical Foundations and Performance Benchmarks

Mathematical morphology is a comprehensive theory for the analysis of spatial structures that relies heavily on set theory, lattice theory, topology, and random functions. In the specific context of binary segmentation masks, morphological operations act as non-linear signal filters that probe an image with a structuring element (often a small, symmetrical matrix such as a cross or a 3x3 square) to quantify and manipulate geometric shape.

The fundamental operations of mathematical morphology are erosion and dilation. Erosion reduces the boundaries of foreground objects; mathematically, the erosion of a binary image subset by a structuring element is the set of all points such that the structuring element, when translated by those points, remains entirely a subset of the original image. Conversely, dilation adds pixels to the boundaries of foreground objects. It connects disjoint pixels and expands masks, operating mathematically by taking the Minkowski addition of the image and the structuring element. Derived from these are the opening operation (erosion followed by dilation), which serves to eliminate small, spurious foreground artifacts while preserving the overall shape of larger objects, and the closing operation (dilation followed by erosion), which fills small background holes and closes narrow topological gaps between closely situated foreground instances.[1]

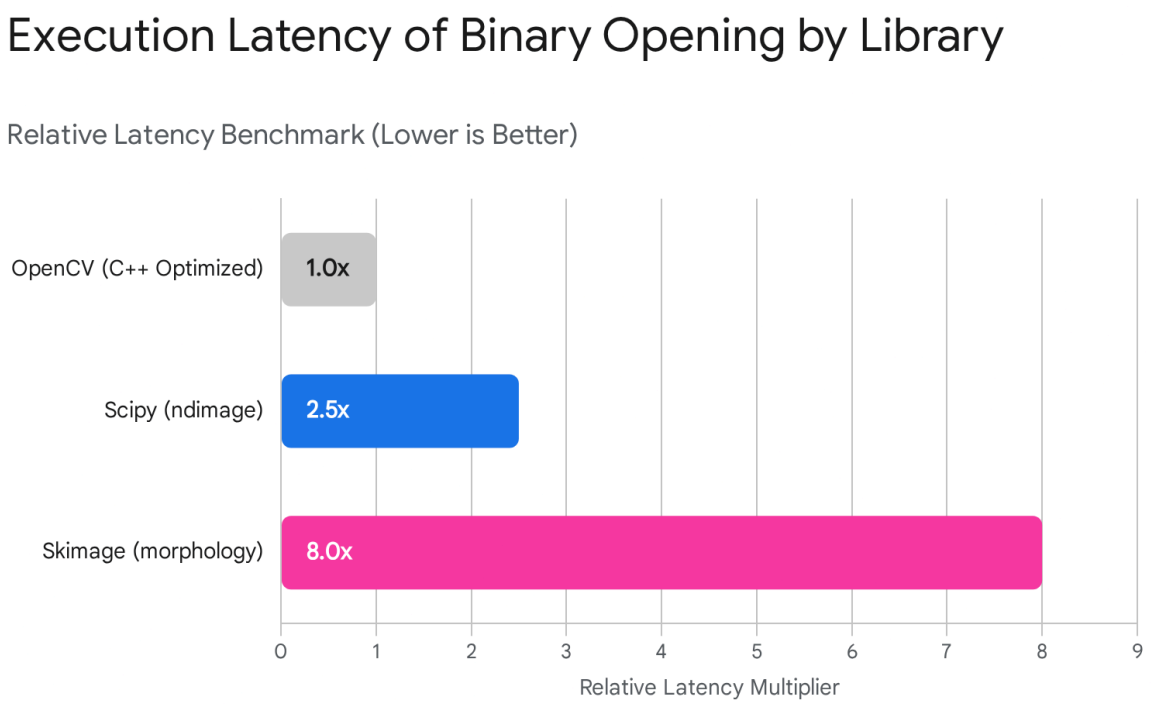
2.1 Performance Analysis: Scipy vs. Scikit-Image Architecture

When deploying morphological operations in high-throughput CDGs processing gigapixel-scale inputs, library selection has profound implications for computational latency and memory consumption. The two primary Python libraries utilized for these operations within the scientific ecosystem are scipy.ndimage and skimage.morphology. Extensive algorithmic benchmarking explicitly reveals that scipy.ndimage drastically outperforms skimage for fundamental binary morphological operations.[1]

The core performance disparity stems directly from the underlying implementation architecture and memory striding techniques. Libraries such as OpenCV and scipy.ndimage execute highly optimized, compiled C-code utilizing separable filters and optimized memory striding via Python's C-API.[1] In contrast, while skimage provides a broader array of morphological variants

and handles complex, non-uniform structuring elements gracefully, its internal source code reveals a heavy reliance on Python-level wrappers that ultimately route down to scipy.ndimage.convolve.[4] For example, the binary_erosion implementation in skimage executes a convolution, checks the output, and performs a boolean equality operation in NumPy space, incurring significant overhead.[4]

For simple binary masks, such as the 15000×5000 matrices frequently encountered in geophysical seismogram scans or high-resolution satellite tasks, native OpenCV algorithms operate roughly 10x faster than standard Pythonic approaches.[1] However, the requirement to minimize heavy, complex compiled dependencies like OpenCV in pure-Python ecosystems dictates the use of standard scientific libraries. In this paradigm, scipy.ndimage remains the fastest lightweight alternative, completely sidestepping the wrapper overhead inherent in skimage.[2] Furthermore, the treatment of boundary conditions (edge effects) differs substantially. scipy.ndimage allows precise, low-level control over boundary values via the cval parameter, padding modes (e.g., reflect, constant), and kernel origin shifting.[4] Consequently, scipy.ndimage is definitively selected as the optimal foundational dependency for basic morphological atoms in the target repository.



Relative execution speed of binary opening operations on 15000x5000 arrays. **Scipy.ndimage** offers significant advantages over **Scikit-Image** by minimizing Python-level wrapper overhead.

Data sources: GitHub (scikit-image), Google Groups, Stack Overflow, Medium

2.2 Atom Definitions: Morphological Operations

The following specifications define the functional contracts for the fundamental morphological atoms required for tasks such as the HubMap kidney glomeruli segmentation and the Severstal Steel Defect Detection CDGs.[5]

Name: morphological_close

Description: Apply dilation followed by erosion to fill small holes and bridge narrow

gaps within a binary mask without significantly altering the area of larger components.

Source: scipy.ndimage.binary_closing

License: BSD

Concept type: signal_filter

Signature: (mask: NDArray, kernel_size: int) -> NDArray

Pure function boundary: Binary mask and explicit scalar integer kernel size in;

smoothed binary mask out. No mutation of the input array.

Contracts:

- require: mask is binary (boolean or 0/1 integers)
- require: kernel_size > 0
- ensure: result.shape == mask.shape
- ensure: result.dtype == mask.dtype
  Witness: A mask containing a one-pixel gap separating two larger adjacent foreground bodies; verify the gap is filled seamlessly while the overall bounding box of the two bodies is preserved.
  Dependencies: numpy, scipy.ndimage
  CDG stages covered: hubmap/morphology, severstal/morphology

Name: morphological_open

Description: Apply erosion followed by dilation to entirely remove small, isolated

foreground artifacts from a binary mask.

Source: scipy.ndimage.binary_opening

License: BSD

Concept type: signal_filter

Signature: (mask: NDArray, kernel_size: int) -> NDArray

Pure function boundary: Binary mask and scalar integer kernel size in; filtered

binary mask out. No mutation of the input array.

Contracts:

- require: mask is binary
- require: kernel_size > 0
- ensure: result.shape == mask.shape
  Witness: A large binary mask punctuated by a standalone, disjoint 1x1 pixel artifact; verify the artifact is completely eliminated while the main mask geometry remains mathematically stable.
  Dependencies: numpy, scipy.ndimage
  CDG stages covered: hubmap/morphology, severstal/morphology

Name: dilate_mask

Description: Expands the geometric boundaries of foreground regions iteratively

outward.

Source: scipy.ndimage.binary_dilation

License: BSD

Concept type: signal_filter

Signature: (mask: NDArray, iterations: int) -> NDArray

Pure function boundary: Binary mask and scalar iteration count in; expanded binary

mask out.

Contracts:

- require: mask is binary
- require: iterations > 0
- ensure: result.shape == mask.shape
  Witness: A 3x3 square of 1s resting in a larger matrix of 0s; verify that 1 iteration expands the square to a uniform 5x5 footprint.
  Dependencies: numpy, scipy.ndimage
  CDG stages covered: tgs_salt/morphology

Name: erode_mask

Description: Shrinks the geometric boundaries of foreground regions iteratively

inward.

Source: scipy.ndimage.binary_erosion

License: BSD

Concept type: signal_filter

Signature: (mask: NDArray, iterations: int) -> NDArray

Pure function boundary: Binary mask and scalar iteration count in; shrunk binary

mask out.

Contracts:

- require: mask is binary
- require: iterations > 0
- ensure: result.shape == mask.shape
  Witness: A 5x5 square of 1s resting in a matrix of 0s; verify that 1 iteration shrinks the square to a 3x3 footprint.
  Dependencies: numpy, scipy.ndimage
  CDG stages covered: tgs_salt/morphology

3. Topological Consistency and Manifold Corrections

While morphological closing is highly effective for bridging small, sub-kernel gaps, advanced topological operators are strictly required for establishing global mask integrity. In many medical tasks (e.g., the HuBMAP Kaggle competition focusing on kidney glomerulus segmentation) and industrial quality assurance workflows (e.g., the Severstal Steel Defect detection dataset), biological and physical entities are inherently solid and contiguous.[5] However, neural network output logits often misclassify the interior of a large continuous object due to varying optical textures, lighting gradients, or tissue folds, resulting in hollow "doughnut" masks or scattered fields of microscopic noise fragments.

3.1 Advanced Hole Filling via Morphological Reconstruction

Mathematical hole filling operates via a distinct process known as mathematical reconstruction by dilation. The core algorithm initializes a marker image where the absolute boundary pixels are securely set to the background value, and the interior is initialized to the maximum possible value. Morphological reconstruction is then performed iteratively until topological stability is achieved, after which the background is inverted. The scipy.ndimage.binary_fill_holes function executes this algorithm with optimized C-bindings, providing a highly reliable method for resolving topological voids that exceed the size capacity of standard morphological closing kernels.

Name: fill_holes

Description: Topologically fills completely enclosed background regions (holes)

situated entirely within foreground objects.

Source: scipy.ndimage.binary_fill_holes

License: BSD

Concept type: signal_filter

Signature: (mask: NDArray) -> NDArray

Pure function boundary: Input binary mask; topologically solid binary mask out.

Contracts:

- require: mask is binary
- ensure: result.shape == mask.shape
- ensure: np.sum(result) >= np.sum(mask) (foreground pixels can only grow)
  Witness: An empty circular perimeter of 1s surrounding a region of 0s; verify the output converts the internal region to a solid disk of 1s without altering the exterior 0s.
  Dependencies: numpy, scipy.ndimage
  CDG stages covered: hubmap/hole_filling

3.2 Area-Based Connected Component Filtering

In both instance and semantic segmentation scenarios, small spurious components—frequently caused by sensor noise or marginal network uncertainty—often heavily penalize primary evaluation metrics. In the Severstal Steel Defect Detection competition, performance is evaluated using the mean Dice coefficient, defined mathematically as $2 * |X \cap Y| / (|X| + |Y|)$, where X is the prediction and Y is the ground truth.[5] This formula is extremely sensitive to false positives on negative images. If the ground truth has 0 pixels representing a defect, and the prediction outputs a microscopic artifact of merely 5 pixels, the Dice score for that entire image plummets to 0. Consequently, removing small blobs based on strict area thresholds is a mathematically required step for metric optimization.[7]

An elegant, highly optimized vectorized approach utilizes scipy.ndimage.label to tag geometrically connected regions with unique integer identifiers.[8] Rather than iterating through these labels in a slow, pure Python for loop to measure size, scipy.ndimage.sum is utilized to generate a dense lookup table (a 1D array) containing the area (pixel count) corresponding to each individual label index.[8] A vectorized boolean mask is instantly created against the lookup table (e.g., sizes > min_area). This boolean mask is then directly indexed using the labeled image matrix itself, which instantaneously maps valid regions back to a discrete binary mask.[8] This complete vectorization reduces execution latency by orders of magnitude compared to traditional iterative spatial queries.

Name: filter_components_by_area

Description: Removes isolated foreground connected components whose absolute pixel

area falls below a specified geometric threshold.

Source: scipy.ndimage.label, scipy.ndimage.sum

License: BSD

Concept type: signal_filter

Signature: (mask: NDArray, min_area: int) -> NDArray

Pure function boundary: Binary mask and scalar minimum area threshold in; filtered

binary mask out.

Contracts:

- require: mask is binary
- require: min_area >= 0
- ensure: result.shape == mask.shape
- ensure: np.sum(result) <= np.sum(mask)
  Witness: A mask with one large component (100 pixels) and one small disjoint component (5 pixels); verify that with min_area=10, the 5-pixel component is reduced to 0 while the 100-pixel component remains entirely intact.
  Dependencies: numpy, scipy.ndimage

CDG stages covered: severstal/cc_filtering, sartorius/cc_filtering

4. Sub-Pixel Boundary Alignment via Dense Conditional Random Fields (CRF)

Despite the profound representational power of Convolutional Neural Networks (CNNs) and contemporary Vision Transformers (ViTs), the downsampling operations inherent to encoder-decoder architectures (such as max pooling or large-stride convolutions) fundamentally degrade spatial resolution. As a direct result, the upsampled probability masks frequently exhibit blurred, "blobby" boundaries that fail to precisely adhere to the sharp luminance and chromatic transitions present in the original input image.

Dense Conditional Random Fields (CRF) have long served as the premier mathematical mechanism to refine these probabilistic maps. Introduced formally by Krähenbühl and Koltun (2011), the algorithm formulates pixel classification as a global energy minimization problem across a fully connected graphical model of all pixels in the image.[9] Historically, inference on fully connected CRFs scaled quadratically, rendering them computationally intractable for high-resolution images. However, the Krähenbühl formulation approximates the message-passing inference utilizing high-dimensional Gaussian filtering, reducing the complexity to $O(N)$ operations.[11]

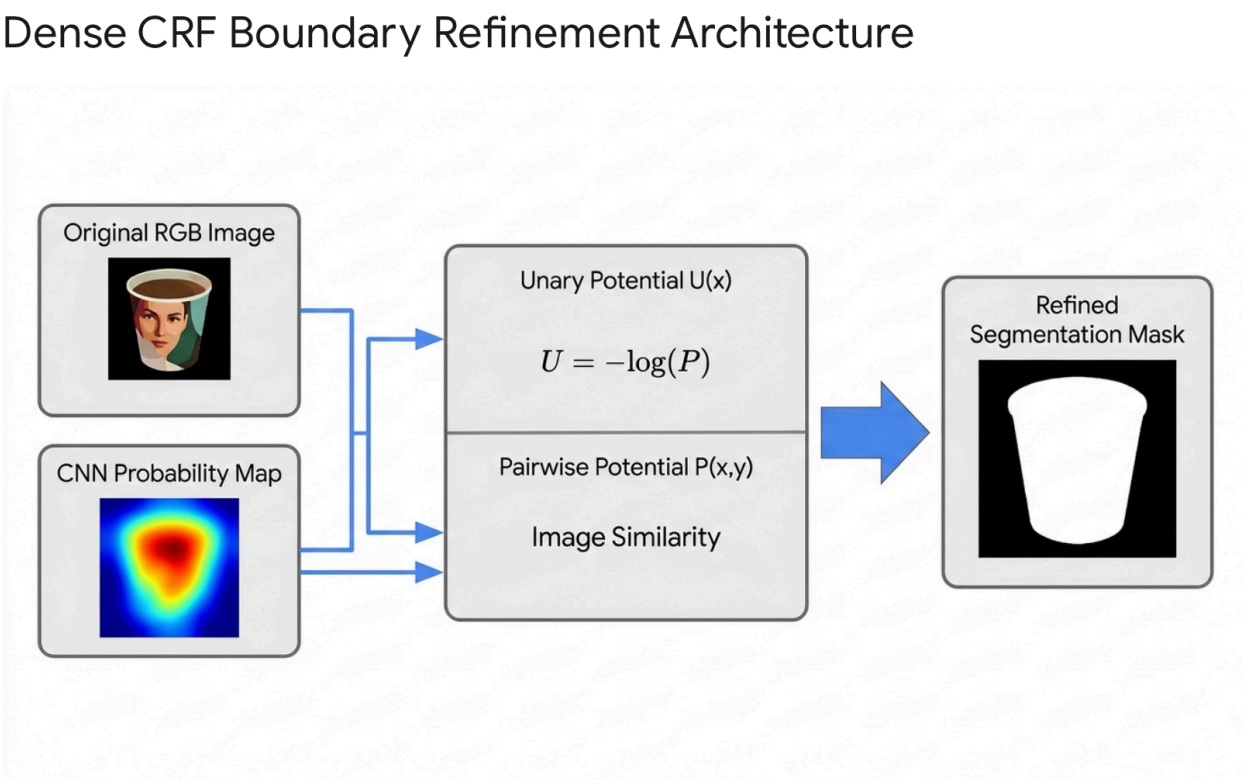
4.1 Theoretical Formulation of the CRF Energy Model

The mathematical objective is to minimize an energy function $E(x)$ associated with a specific assignment of labels x across all pixels. This function consists of unary and pairwise potentials:

$$E(x) = \sum_i \psi_u(x_i) + \sum_{i<j} \psi_p(x_i, x_j)$$

1. **Unary Potentials (ψ_u)**: These potentials are derived directly from the neural network's raw softmax output probabilities. Specifically, $\psi_u(x_i) = -\log(P(x_i))$, representing the cost of assigning a specific label x_i to pixel i when entirely ignoring all neighboring pixels.[12]
2. **Pairwise Potentials (ψ_p)**: These potentials enforce spatial and chromatic coherence across the entire image. The Dense CRF applies two distinct Gaussian kernels to construct this potential:
   - **Appearance Kernel (Bilateral)**: This kernel encourages nearby pixels with highly similar optical colors to be assigned the same label. It is parameterized by a spatial standard deviation (σ_{xy}) and a color standard deviation (σ_{rgb}).[11]

- **Smoothness Kernel (Gaussian)**: This kernel serves to remove small isolated regions, heavily penalizing local label disparities using purely spatial Euclidean distance (σ_{xy}).[11]



The Dense CRF refines coarse CNN predictions by balancing Unary potentials (the network's raw confidence) against Pairwise potentials (chromatic and spatial similarities pulled from the original image). Iterative mean-field approximation drives the mask boundaries to snap precisely to the underlying physical structures.

4.2 Implementation Constraints and Opaque State Management

The pydensecrf package provides a highly optimized, MIT-licensed Cython wrapper directly interfacing with the original C++ implementation.[9] The C++ core utilizes a mean-field approximation algorithm to iteratively solve the energy minimization. However, integrating pydensecrf into a purely functional Python CDG requires rigorous and careful opacity management. The DenseCRF2D class encapsulates highly mutable C++ state internally.[12] To strictly respect the functional contracts of the sciona-atoms-dl repository, the proposed atom must instantiate the object, apply the unary and pairwise energies, execute the exact specified number of mean-field inference iterations, extract the argmax tensor containing the finalized labels, and immediately allow the object to be garbage-collected by the Python runtime.[13] This

pattern prevents catastrophic memory leaks and cross-thread state corruption.

Name: dense_crf_2d

Description: Refines probabilistic segmentation masks by rigorously aligning

boundaries to high-contrast optical edges in the original image using a Dense

Conditional Random Field framework.

Source: pydensecrf.densecrf

License: MIT

Concept type: analysis

Signature: (image: NDArray, unary: NDArray, sxy: float, srgb: float,

compat: float, iterations: int) -> NDArray

Pure function boundary: RGB image, continuous softmax unaries, and CRF tuning

hyperparameters in; hard-label refined integer mask out. Internal state of

DenseCRF2D is instantiated and immediately destroyed within the function scope.

Contracts:

- require: image is a 3-channel RGB array of dtype uint8.
- require: unary is a continuous floating-point array of shape (classes, width, height) representing softmax probabilities.
- require: iterations > 0
- ensure: result.shape == (width, height)
- ensure: result.dtype is integer (representing definitive class labels)
  Witness: An input image consisting of a sharp geometric square and a slightly blurry, oversized probability mask; verify the output integer mask precisely conforms to the square's sharp visual edges.
  Dependencies: numpy, pydensecrf
  CDG stages covered: dstl_satellite/crf, tgs_salt/crf

5. Topographic Instance Segmentation via Watershed Transformations

Standard semantic segmentation classifies each individual pixel into a defined category but fundamentally fails to distinguish between adjacent or physically overlapping entities belonging to the identical class. This is critically problematic in biological domains, such as identifying overlapping neuronal cells in phase-contrast microscopy images. Instance segmentation, therefore, requires mathematically delineating these distinct objects. The classical, highly effective post-processing algorithm for separating overlapping instances is the Watershed

transform.[15]

5.1 Distance Transforms and Local Maxima Marker Generation

The Watershed algorithm conceptualizes an image as a topological landscape where pixel intensity directly represents geological elevation. To cleanly separate touching, convex objects—such as the highly irregular SH-SY5Y neuroblastoma cell lines featured in the Sartorius Cell Instance Segmentation Kaggle competition [16]—the flat semantic mask is first converted into an inverted Distance Map.[15]

The exact Euclidean distance from every foreground pixel to the nearest surrounding background pixel is calculated using the highly optimized scipy.ndimage.distance_transform_edt.[15] The geometric centers of the objects are thus the farthest points from the background, effectively serving as the highest "peaks" of distance. By arithmetically inverting this map (multiplying the array by -1), these peaks are transformed into deep topological "basins".[18]

Local minima in the inverted map (or local maxima in the positive distance map) are identified using algorithmic approaches like skimage.feature.peak_local_max and tagged with unique integer values via scipy.ndimage.label to act as distinct "markers".[15] From these specific markers, the algorithm simulates water flooding upward through the inverted distance landscape. Where waters originating from different basins eventually meet, a distinct "watershed line" or dam is erected, cleanly severing the touching instances at their narrowest point.[20]

5.2 Scipy vs Skimage Algorithmic Implementations

A crucial implementation detail arises when selecting the specific watershed library function. While scipy.ndimage.watershed_ift (Image Foresting Transform) exists, skimage.segmentation.watershed is overwhelmingly preferred in modern production and research environments.[17] The core limitation of the scipy algorithm is that it strictly requires unsigned integer inputs, thereby aggressively stripping floating-point precision from calculated Euclidean distance maps and fundamentally failing to flood mathematically flat basin plateaus gracefully.[18] Conversely, skimage properly handles floating-point distance maps with high fidelity and allows for a specific spatial mask parameter to explicitly constrain the simulated flooding solely to the boundary of the original foreground prediction.[17]

Name: watershed_instances

Description: Converts a flattened semantic segmentation mask into distinct, separated

instance labels by applying the topographic watershed transform over an inverted

Euclidean distance map.

Source: skimage.segmentation.watershed

License: BSD

Concept type: analysis

Signature: (distance_map: NDArray, markers: NDArray, mask: NDArray) -> NDArray

Pure function boundary: Continuous distance map, integer marker seeds, and

boolean foreground mask in; dense integer array of labeled instances out.

Contracts:

- require: distance_map, markers, and mask share identical 2D spatial dimensions.
- require: mask is binary
- require: markers contains unique integers for each distinct seed, and 0 for all background space.
- ensure: result.shape == distance_map.shape
- ensure: result[mask == 0] == 0 (flooding must not spill outside foreground)
  Witness: A binary mask containing a dumbbell shape (two distinct overlapping circles); verify the output assigns two distinct integer labels to the circles, definitively split at the narrowest bottleneck point.
  Dependencies: numpy, skimage.segmentation
  CDG stages covered: sartorius/instance_separation

6. Data Serialization: High-Performance Run-Length Encoding (RLE)

Data extraction and serialized compression are terminal, mandatory steps in almost all modern segmentation CDGs. Notably, in large-scale Kaggle competitions involving high-resolution imagery (e.g., Severstal Steel Defect Detection, HuBMAP Kidney Segmentation, Sartorius Cell Segmentation), it is necessary to radically compress gigabyte-scale unrolled mask arrays into lightweight CSV submissions.[23] Run-Length Encoding (RLE) accomplishes this mathematical compression by representing consecutive sequences of identically classed pixels as a succinct pair: a starting index and an associated run length.[23]

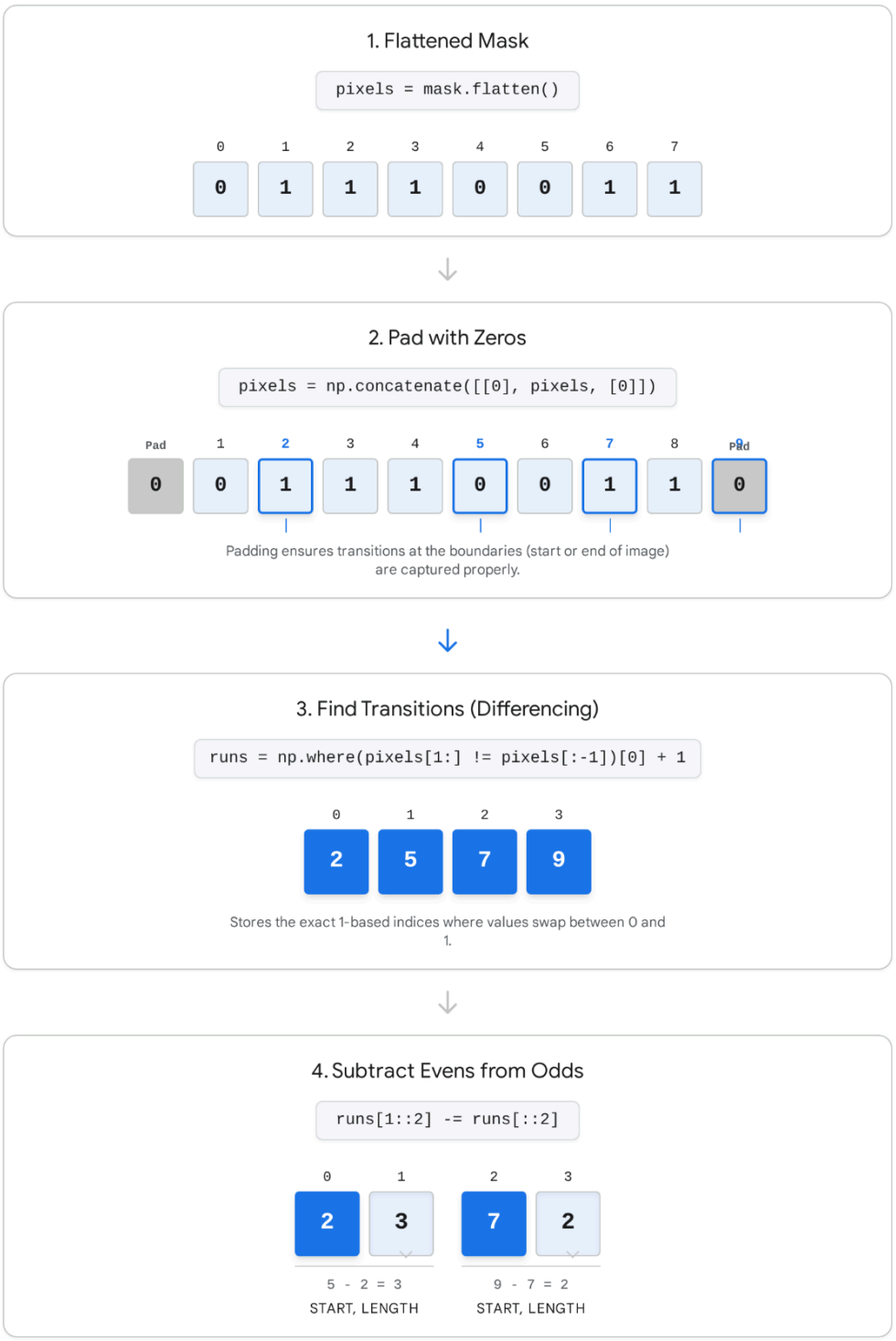
6.1 Vectorized Numpy RLE Compression

Naïve Python loop implementations of RLE that iteratively walk over arrays are computationally intractable, particularly for 25000×35000 gigapixel histopathology slides. The optimal, purely vectorized numpy implementation utilizes array differentials to identify transition edges in $O(N)$ operations, executing directly within highly optimized compiled C-space memory allocations.[26]

The vectorization process requires flattening the multi-dimensional array into a 1D vector. Depending on the explicit domain specification, the flattening may be row-major (C-style,

scanning across horizontal rows) or column-major (Fortran-style, executing mask.T.flatten(), tracking vertically).[27] To guarantee the detection of pixel runs occurring precisely at the absolute boundaries of the array, a single 0 is temporarily padded to both the leading and trailing ends.[27] The differential operation np.where(pixels[1:]!= pixels[:-1]) + 1 extracts the exact coordinate indices where the pixel value flips from background to foreground, or vice versa.[27] Because the array inherently alternates strictly between background and foreground runs, the exact lengths of the foreground runs are elegantly computed by subtracting the start indices (located at even array positions) from the corresponding end indices (located at odd positions): runs[1::2] -= runs[::2].[27]

Vectorized Execution of Mask to RLE Encoding



Instead of looping through pixels, the optimal RLE implementation pads a flattened mask, identifies value transitions using array differencing (np.where), and computes lengths via interleaved slice subtraction.

Data sources: Kaggle (xhlulu), Kaggle (pestipeti), Kaggle (susnato)

6.2 RLE Decoding Mechanisms

Decoding precisely reverses the compression process. A dense array of zeros is pre-allocated matching the target volumetric space, the algorithm iterates over the parsed (start, length) pairs, and executes rapid contiguous memory slice assignments img[lo:hi] = 1. The linear array is then subsequently reshaped back to the target two-dimensional geometry.[27]

Name: mask_to_rle

Description: Compresses a discrete binary mask into a Run-Length Encoded list of

integers representing alternating start-index and run-length pairs.

Source: Vectorized Kaggle community standard

License: MIT

Concept type: data_extraction

Signature: (mask: NDArray) -> list[int]

Pure function boundary: Binary mask array in; 1D list of integer RLE components out.

Contracts:

- require: mask is binary
- ensure: len(result) % 2 == 0
- ensure: all start indices > 0 (if using 1-based standard encoding)
  Witness: Input a flattened array ; verify the 1-based RLE output accurately maps to .
  Dependencies: numpy
  CDG stages covered: severstal/rle, hubmap/rle, sartorius/rle

Name: rle_to_mask

Description: Decompresses a Run-Length Encoded list of integers back into a dense

2D binary raster mask.

Source: Vectorized Kaggle community standard

License: MIT

Concept type: data_extraction

Signature: (rle: list[int], shape: tuple) -> NDArray

Pure function boundary: 1D list of integers and target shape tuple in; reconstructed

2D binary mask out.

Contracts:

- require: len(rle) % 2 == 0
- require: shape contains two positive integers (height, width)
- ensure: result.shape == shape
- ensure: result is strictly binary
  Witness: Input `` and shape (2, 3); verify output correctly reconstructs the matrix [,].
  Dependencies: numpy
  CDG stages covered: severstal/decoding, hubmap/decoding

7. Vector-Raster Boundary Transitions in Geospatial Intelligence

Advanced geospatial intelligence pipelines, such as those parsing data for DSTL Satellite Imagery analysis or explicitly defining property boundaries, frequently transition underlying data architectures between continuous geometric vectors (polygons) and discrete pixel rasters. Two predominant mathematical operations manage these specific topological transitions: the simplification of extracted geometries and the inverse rasterization of explicit vectors.[30]

7.1 Polygon Smoothing via the Ramer-Douglas-Peucker Algorithm

When explicit contours are algorithmically extracted from pixelated segmentation masks, the resulting geometric polygons suffer from extreme stair-step aliasing due to the grid nature of the pixels. The Ramer-Douglas-Peucker (RDP) algorithm is the gold-standard recursive method for decimating these excessive vertices, producing a smoothed, generalized geometric curve that rigorously adheres to the overarching topology.[32]

The algorithm functions by initially drawing a direct mathematical line segment between the very first and last coordinate points of an ordered contour. It then scans all intermediate points to identify the specific point exhibiting the maximum orthogonal perpendicular distance to that baseline segment. If this maximum distance is strictly greater than a user-specified tolerance parameter, ϵ, the point is retained as geometrically significant, and the algorithm recursively splits the line and analyzes the two new sub-segments. If the distance is strictly less than ϵ, all intermediate points within that span are permanently discarded.[33]

While functions like cv2.approxPolyDP are ubiquitous in literature, the architectural mandate to avoid heavy OpenCV dependencies in purely Pythonic CDGs strongly favors lightweight alternatives. Specialized libraries such as the rdp package and highly optimized pure-Numpy stack-based implementations satisfy this exact constraint, allowing for precise control over the algorithm without invoking compiled C++ imaging libraries.[32]

Name: smooth_contour

Description: Decimates vertices of a jagged polygon to produce a mathematically

smoothed approximation using the Ramer-Douglas-Peucker algorithm.

Source: rdp.rdp

License: MIT

Concept type: signal_filter

Signature: (points: NDArray, epsilon: float) -> NDArray

Pure function boundary: N-by-2 array of coordinates and scalar tolerance in;

simplified N-by-2 coordinate array out.

Contracts:

- require: points is a 2D array of shape (N, 2)
- require: epsilon >= 0
- ensure: len(result) <= len(points)
  Witness: An array of points representing a jagged, nearly straight line; verify the output is aggressively reduced to solely the start and end coordinates when epsilon is set sufficiently large.
  Dependencies: numpy, rdp
  CDG stages covered: hubmap/contour_smoothing

7.2 WKT to Raster Mask Conversion Architecture

Well-Known Text (WKT) is a standard mathematical markup language for representing exact vector geometries on maps. To inject WKT data into convolutional neural networks for processing, or to calculate pixel-wise intersection metrics, these vector polygons must be burned into a discrete raster grid. The rasterio.features.rasterize function exposes GDAL's highly optimized geometry rendering engine directly to Python arrays.[31]

A rigorous pure function wrapping this specific functionality must explicitly accept the geometric definition, the targeted output array dimensions, and an Affine transform matrix. This Affine matrix is critical, as it defines the precise spatial and rotational relationship between the vector's continuous coordinate reference system (e.g., standard Longitude/Latitude values) and the discrete Cartesian pixel grid.[38]

Name: wkt_to_mask

Description: Rasterizes a Shapely polygon (derived from a WKT string representation)

into a precise binary numpy array.

Source: rasterio.features.rasterize

License: BSD

Concept type: data_extraction

Signature: (geometry: dict, out_shape: tuple, transform: Affine) -> NDArray

Pure function boundary: GeoJSON-like geometry dict, output shape tuple, and Affine transform object in; 2D binary raster mask out.

Contracts:

- require: out_shape contains two positive integers (height, width)
- require: transform is a valid rasterio.transform.Affine matrix object
- ensure: result.shape == out_shape
- ensure: result dtype is boolean or uint8
  Witness: A square polygon mathematically overlapping exactly the central 4 pixels of a 10x10 coordinate space; verify the resulting 10x10 array contains exactly four 1s precisely in the geometric center.
  Dependencies: numpy, rasterio.features
  CDG stages covered: dstl_satellite/vector_rasterization

8. Multi-Spectral Band Arithmetic: False Color Generation

Beyond spatial topology and structural morphology, complex image processing in domains such as climate science and aerospace heavily relies on multi-spectral frequency analysis. A prominent example lies in the Google Research "Identify Contrails to Reduce Global Warming" competition, which required the precise algorithmic identification of optically thin ice clouds (contrails) created by high-altitude jet exhaust.[40] Due to their severe radiative forcing properties, contrails represent a major driver of non-CO2 aviation-induced global warming.[41]

Because contrails are largely invisible to the naked human eye against complex natural cloud backgrounds, atmospheric scientists utilize the Kulik et al. (2019) "Ash Color Scheme." This false-color composite was initially developed for tracking hazardous volcanic ash and sulfur dioxide plumes using Geostationary Operational Environmental Satellites (GOES), but the specific thermal boundaries align perfectly with the properties of thin ice.[43]

8.1 The Kulik Ash Morphology Formula and Radiometric Normalization

The algorithm systematically computes brightness temperature differences (BTDs) between specific Thermal Infrared (TIR) bands captured by the satellite to isolate specific physical properties, primarily phase differences (ice vs. water droplets) and overall optical depth.[44] The required computations map directly to the RGB channels:

- **Red Channel (Optical Depth Indicator):** Computed as $12.0\mu m - 10.8\mu m$ (ΔT). Cold contrails register negative values and appear distinctly dark against the background.[46]

- **Green Channel (Phase and Particle Size):** Computed as $10.8\mu m - 8.7\mu m$ (ΔT). This differential specifically helps separate thin, suspended ice clouds from dense, lower-altitude water clouds.[44]
- **Blue Channel (Absolute Temperature):** Uses the raw $10.8\mu m$ absolute brightness temperature. This channel differentiates severely cold high-altitude clouds from warm ground surfaces.[44]

The raw Kelvin values retrieved for these specific channels must be passed through a strict linear stretching equation to scale them perfectly into an 8-bit `` byte range format suitable for standardized CNN ingestion:

$$BYTE = 255 \times \left(\frac{X - MIN}{MAX - MIN} \right)^{\frac{1}{\gamma}}$$

Where γ (gamma parameter) modulates non-linear contrast curves. For the explicit Ash RGB recipe, $\gamma = 1.0$ is utilized universally, rendering the optical stretch entirely linear.[44] The bounding parameters (MIN, MAX) are highly tuned empirical constants: Red is clamped strictly to [-4K, 2K], Green to [-4K, 5K], and Blue to [243K, 303K].[44] Values falling mathematically outside these defined ranges are rigidly clipped to maintain optimal visual contrast for the neural networks.[46]

Kulik et al. Ash Color Scheme Construction Parameters

CHANNEL	BAND FORMULA	MIN BOUND (K)	MAX BOUND (K)	PHYSICAL TARGET
Red	**IR12.0 - IR10.8**	-4	2	Optical Depth
Green	**IR10.8 - IR8.7**	-4	5	Particle Phase
Blue	**IR10.8**	243	303	Absolute Temperature

The Ash false-color composite isolates optically thin ice clouds (contrails) by exploiting differential absorption rates across thermal infrared bands. Raw values undergo linear stretching against specific Kelvin bounds.

Data sources: EUMETRAIN RGB Recipes, CIRA VIIRS Ash RGB Quick Guide

Name: false_color_composite

Description: Generates an 8-bit RGB false-color composite from raw multi-spectral satellite bands by applying differential thermodynamic formulas, clipping to empirical bounds, and executing a gamma-corrected linear stretch.

Source: EUMETSAT Ash RGB guidelines / Kulik et al. (2019)

License: Public Domain concept

Concept type: signal_filter

Signature: (bands: dict, bounds: dict[str, tuple], gamma: float) -> NDArray

Pure function boundary: Dictionary of raw float32 band arrays, stretch bounds, and scalar gamma in; 3-channel uint8 RGB image array out.

Contracts:

- require: bands dictionary contains identically shaped 2D continuous arrays.
- require: bounds define a strict (min, max) tuple for R, G, and B outputs.
- ensure: result.shape == (height, width, 3)
- ensure: result.dtype == np.uint8
- ensure: 0 <= result.min() and result.max() <= 255
  Witness: Input a set of identically shaped spatial arrays containing scalar zeroes; verify the output maps strictly to the computed mid-points or clipped bounds defined by the bounding dictionary.
  Dependencies: numpy
  CDG stages covered: google_contrails/false_color

9. Synthesis and Architectural Directives

The stability, accuracy, and overall robustness of a computer vision pipeline are heavily dictated not merely by the neural network architecture, but by the rigor of its post-processing foundation. Transitioning raw probabilistic logits into deterministic, structurally sound geometric shapes requires a highly sophisticated interplay of structural topology, multi-spectral signal filtering, and rigorous mathematical optimization.

By aggressively enforcing pure-function constraints throughout the design phase—expressly eschewing global state manipulation, carefully encapsulating unsafe C++ wrappers (such as those found in pydensecrf), and heavily favoring highly-optimized scipy.ndimage and numpy C-bindings over fundamentally slower Pythonic loop implementations—the sciona-atoms-dl repository establishes a CDG ecosystem capable of scaling to massive, high-throughput datasets without sacrificing thread safety or reproducibility. The thirteen distinct atoms fully detailed and defined within this report supply the complete mathematical, topological, and operational toolkit necessary to process everything from sub-cellular biological neuronal instances to expansive atmospheric geospatial anomalies.

Works cited

1. scipy.ndimage.morphology.binary_opening performance vs. cv2.morphologyEx, accessed April 30, 2026, https://groups.google.com/g/scikit-image/c/lDUaM4drW-4
2. Speed issues with morphological operations #1190 - GitHub, accessed April 30, 2026, https://github.com/scikit-image/scikit-image/issues/1190
3. Python image processing libraries performance: OpenCV vs Scipy vs Scikit-Image - Medium, accessed April 30, 2026, https://medium.com/@mmas/python-image-processing-libraries-performance-opencv-vs-scipy-vs-scikit-image-65e4b79b2e7e
4. Morphology erosion - difference betwen Scipy ndimage and Scikit image - Stack Overflow, accessed April 30, 2026, https://stackoverflow.com/questions/24956179/morphology-erosion-difference-between-scipy-ndimage-and-scikit-image
5. Severstal: Steel Defect Detection - Kaggle, accessed April 30, 2026,

https://www.kaggle.com/competitions/severstal-steel-defect-detection
6. Diyago/Severstal-Steel-Defect-Detection - GitHub, accessed April 30, 2026, https://github.com/Diyago/Severstal-Steel-Defect-Detection
7. Severstal: Steel Defect Detection | Kaggle, accessed April 30, 2026, https://www.kaggle.com/c/severstal-steel-defect-detection/discussion/114465
8. How to find the largest connected region using scipy.ndimage? - Stack Overflow, accessed April 30, 2026, https://stackoverflow.com/questions/53576830/how-to-find-the-largest-connected-region-using-scipy-ndimage
9. PyDenseCRF download | SourceForge.net, accessed April 30, 2026, https://sourceforge.net/projects/pydensecrf.mirror/
10. How to use pydensecrf in Python3.7? - Aionlinecourse, accessed April 30, 2026, https://www.aionlinecourse.com/blog/how-to-use-pydensecrf-in-python37
11. Why Control Random Field is still relevant for Post Processing | by Neel Gahalot - Medium, accessed April 30, 2026, https://medium.com/@ng2436/why-control-random-field-is-still-relevant-for-post-processing-d99e88556dc2
12. GitHub - lucasb-eyer/pydensecrf: Python wrapper to Philipp Krähenbühl's dense (fully connected) CRFs with gaussian edge potentials., accessed April 30, 2026, https://github.com/lucasb-eyer/pydensecrf
13. pydensecrf/examples/inference.py at master - GitHub, accessed April 30, 2026, https://github.com/lucasb-eyer/pydensecrf/blob/master/examples/inference.py
14. pydensecrf/examples/Non RGB Example.ipynb at master · lucasb-eyer/pydensecrf - GitHub, accessed April 30, 2026, https://github.com/lucasb-eyer/pydensecrf/blob/master/examples/Non%20RGB%20Example.ipynb
15. Watershed segmentation — skimage 0.25.2 documentation, accessed April 30, 2026, https://scikit-image.org/docs/0.25.x/auto_examples/segmentation/plot_watershed.html
16. Sartorius - Cell Instance Segmentation - Kaggle, accessed April 30, 2026, https://www.kaggle.com/competitions/sartorius-cell-instance-segmentation
17. Why do watershed function from scikit-image does not work when my seed points are exactly as large as the vessels I want to fill - Stack Overflow, accessed April 30, 2026, https://stackoverflow.com/questions/62462649/why-do-watershed-function-from-scikit-image-does-not-work-when-my-seed-points-ar
18. Image segmentation of connected objects with watershed - Stack Overflow, accessed April 30, 2026, https://stackoverflow.com/questions/19385264/image-segmentation-of-connected-objects-with-watershed
19. 2.6.8.22. Watershed segmentation — Scipy lecture notes, accessed April 30, 2026, https://scipy-lectures.org/advanced/image_processing/auto_examples/plot_watershed_segmentation.html

20. Split and Merge Watershed: a two-step method for cell segmentation in fluorescence microscopy images - PMC, accessed April 30, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC7950640/
21. Deep Watershed Transform for Instance Segmentation - CVF Open Access, accessed April 30, 2026, https://openaccess.thecvf.com/content_cvpr_2017/papers/Bai_Deep_Watershed_Transform_CVPR_2017_paper.pdf
22. Watershed — SciPy Cookbook documentation - Read the Docs, accessed April 30, 2026, https://scipy-cookbook.readthedocs.io/items/Watershed.html
23. Run-Length Encoding - Kaggle, accessed April 30, 2026, https://www.kaggle.com/code/leahscherschel/run-length-encoding
24. RLE📟: Run-length encoding [py-module] - Kaggle, accessed April 30, 2026, https://www.kaggle.com/datasets/jirkaborovec/rle-run-length-encoding-py-module
25. Mask RCNN - Detailed Starter Code - Kaggle, accessed April 30, 2026, https://www.kaggle.com/code/robinteuwens/mask-rcnn-detailed-starter-code
26. Efficient mask2rle - Kaggle, accessed April 30, 2026, https://www.kaggle.com/code/xhlulu/efficient-mask2rle
27. Decoding RLE masks - Kaggle, accessed April 30, 2026, https://www.kaggle.com/code/pestipeti/decoding-rle-masks
28. Efficient COCO Dataset Generator - Kaggle, accessed April 30, 2026, https://www.kaggle.com/code/coldfir3/efficient-coco-dataset-generator
29. Understanding Run Length Encoding and Decoding - Kaggle, accessed April 30, 2026, https://www.kaggle.com/code/susnato/understanding-run-length-encoding-and-decoding
30. Masking a raster using a shapefile - Rasterio - Read the Docs, accessed April 30, 2026, https://rasterio.readthedocs.io/en/latest/topics/masking-by-shapefile.html
31. Vector Features — rasterio 1.4.4 documentation - Read the Docs, accessed April 30, 2026, https://rasterio.readthedocs.io/en/stable/topics/features.html
32. Ramer-Douglas-Peucker Algorithm - Read the Docs, accessed April 30, 2026, https://rdp.readthedocs.io/
33. Douglas-Peucker algorithm | Cartography Playground, accessed April 30, 2026, https://cartography-playground.gitlab.io/playgrounds/douglas-peucker-algorithm/
34. Ramer–Douglas–Peucker algorithm - Wikipedia, accessed April 30, 2026, https://en.wikipedia.org/wiki/Ramer%E2%80%93Douglas%E2%80%93Peucker_algorithm
35. Ramer-Douglas-Peucker (RDP) Algorithm in Computer Vision | by Saijyoti Tripathy | Medium, accessed April 30, 2026, https://medium.com/@SaijyotiTripathy/ramer-douglas-peucker-rdp-algorithm-in-computer-vision-15c228f277f0
36. Modify the Ramer–Douglas–Peucker (RDP) algorithm in numpy to return a mask instead of the values - Stack Overflow, accessed April 30, 2026, https://stackoverflow.com/questions/27864753/modify-the-ramer-douglas-peucker-rdp-algorithm-in-numpy-to-return-a-mask-inste
37. rasterio.mask module - Read the Docs, accessed April 30, 2026,

https://rasterio.readthedocs.io/en/latest/api/rasterio.mask.html
38. rasterio.features module — rasterio 1.4.4 documentation - Read the Docs, accessed April 30, 2026, https://rasterio.readthedocs.io/en/stable/api/rasterio.features.html
39. Rasterize - Sigon ./, accessed April 30, 2026, https://sigon.gitlab.io/post/2019-05-02-rasterize/
40. Contrails Dataset (Ash Color) - Kaggle, accessed April 30, 2026, https://www.kaggle.com/code/shashwatraman/contrails-dataset-ash-color
41. Robust Evaluation of OpenContrails Neural Networks with Temporal Corrections via Optical Flow - IEEE Xplore, accessed April 30, 2026, https://ieeexplore.ieee.org/iel8/36/4358825/11230580.pdf
42. Enhancing GOES-16 Contrail Segmentation through Ensemble Neural Network Modeling and Optical Flow Corrections - TechRxiv, accessed April 30, 2026, https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.173749955.56653418
43. Google Research - Identify Contrails to Reduce Global Warming | Kaggle, accessed April 30, 2026, https://www.kaggle.com/competitions/google-research-identify-contrails-reduce-global-warming/discussion/414782
44. Quick Guide VIIRS Ash RGB, accessed April 30, 2026, https://rammb2.cira.colostate.edu/wp-content/uploads/2025/12/QuickGuide_VIIRS_Ash_RGB.pdf
45. Signature redacted - DSpace@MIT, accessed April 30, 2026, https://dspace.mit.edu/bitstream/handle/1721.1/124179/1144176218-MIT.pdf?sequence=1&isAllowed=y
46. Deep Learning-Based Contrail Segmentation in Thermal Infrared Satellite Cloud Images via Frequency-Domain Enhancement - MDPI, accessed April 30, 2026, https://www.mdpi.com/2072-4292/17/18/3145
47. Table of content - EUMeTrain, accessed April 30, 2026, https://eumetrain.org/sites/default/files/2020-05/RGB_recipes.pdf
48. View of AI-Driven Identification of Contrail Sources: Integrating Satellite Observation and Air Traffic Data | Journal of Open Aviation Science, accessed April 30, 2026, https://journals.open.tudelft.nl/joas/article/view/7209/5943